Attack Simulation Report — Password Spraying (FTP via Hydra)

1. Attack Overview

This report documents a simulated password spraying attack against the Metasploitable2 FTP service, using Hydra in list-mode to target many usernames with a single fixed password — the mirror image of the earlier FTP brute-force attack, which targeted one username with many passwords.

Field	Detail
Attack type	Password spraying (multi-account, single-password credential guessing)
Tool	Hydra v9.7
Attacker	Kali Linux (10.0.2.15)
Target	Metasploitable2 vsftpd service (10.0.2.3, port 21)
Username list	7 plausible accounts (msfadmin, admin, root, user, service, postgres, ftp)
MITRE ATT&CK	T1110.003 (Password Spraying)
Detection rule	Existing rule 100017 fires but mislabels the attack; custom rule 100020 attempted, documented limitation

Objective: demonstrate that an existing frequency-based rule, written for classic brute force, correctly alerts on spraying traffic too — but mislabels it — and attempt a purpose-built rule to distinguish the two attack shapes correctly.

2. Attack Execution

A short username list was created on Kali, then Hydra was run in list-mode (`-L`) against a single fixed password (`-p`) — the inverse of the original FTP brute-force command, which used one username (`-l`) against a password list:

```
hydra -L userlist.txt -p Summer2026! ftp://10.0.2.3 -t 4 -V
```

The attack was run twice over the course of this exercise, each with a different fixed password and thread count, to generate independent, clean spray traffic for testing the new detection rule:

1. **Run 1** (`-t 4`, `Summer2026!`) — initial spray, used to confirm the existing rule 100017's behavior against this attack shape.
2. **Run 2** (`-t 1`, `Winter2026!`) — single-threaded re-run, used to rule out thread interleaving as a cause once the custom rule didn't fire on Run 1.

3. Raw Log Verification

Searching Discover for `vsftpd` over the attack window confirmed the spray pattern directly: each `PAM: User login failed.` event (rule 5503) carried a **different** `data.dstuser` value — `postgres`, `admin`,

`root`, `user`, `service` — unlike the original FTP brute-force attack, where every failed attempt targeted the same account (`msfadmin`) repeatedly.

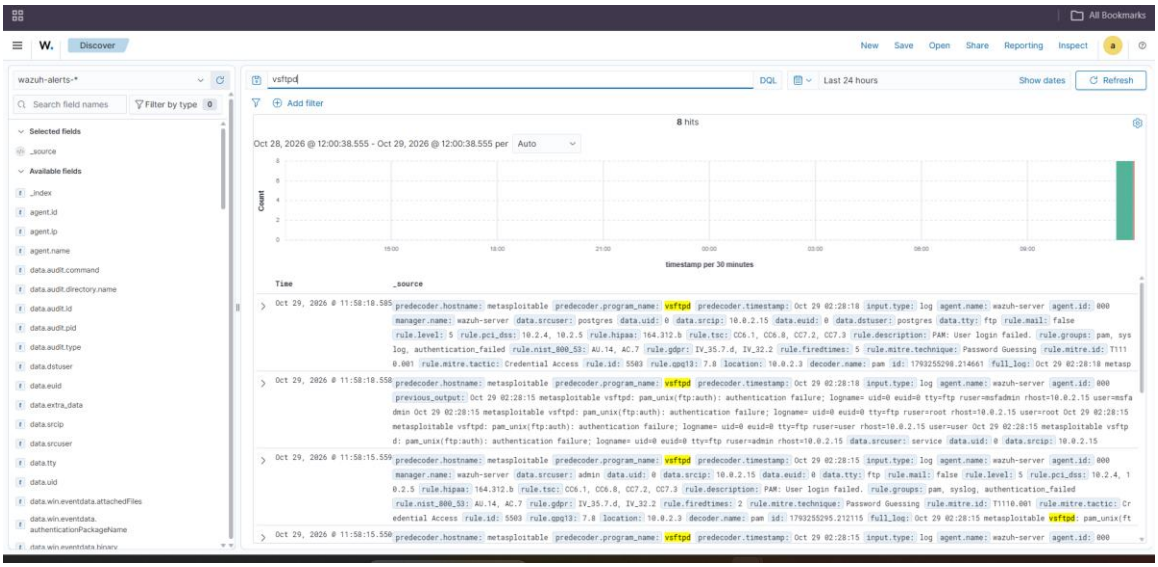


Figure 1: Discover, query `vsftpd`, Last 24 hours — 8 hits with distinct `data.dstuser` values (`postgres`, `admin`, `root`, `user`, `service`) confirming the attack targeted multiple accounts from one source, rather than repeatedly guessing against one.

A later run's `previous_output` field captured the full sequence cleanly: `msfadmin → admin → root → user → service → postgres` — six consecutive, entirely distinct usernames, no repeats anywhere in the chain. This clean sequence became important later when ruling out attack-tool retry behavior as an explanation for the custom rule's behavior (Section 6).

4. Detection Analysis — Correct Alert, Wrong Label

Checking Threat Hunting → Events (`rule.level >= 10`) immediately after the first spray run showed a high-severity alert firing — but under the existing FTP brute-force rule, not anything specific to spraying:

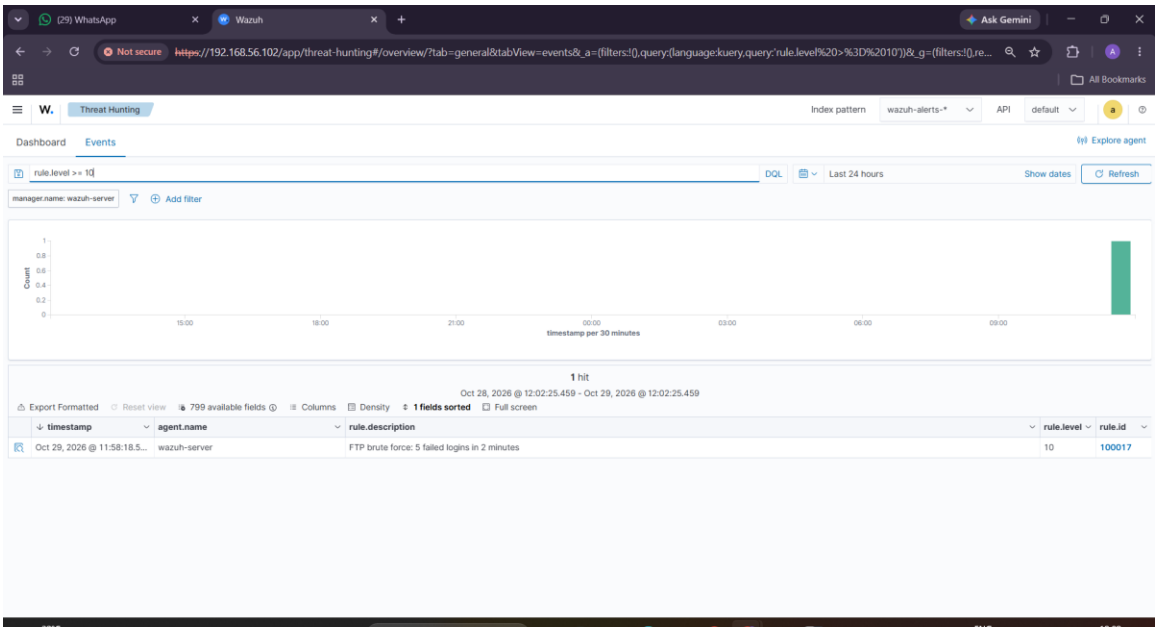


*Figure 2: Threat Hunting → Events, rule.level >= 10 — rule **100017** fired on the spray traffic, labeled "FTP brute force: 5 failed logins in 2 minutes."*

Finding: rule 100017's logic (`if_matched_sid: 5503`, `same_source_ip`, `frequency: 5`, `timeframe: 120`) only checks that five PAM failures came from one source within the window — it does not check whether those failures targeted one account or several. The rule correctly fired and correctly flagged the activity as suspicious, but its description would mislead an analyst into assuming a single account was being hammered, when in fact seven different accounts were probed. In a real SOC, this distinction matters: brute force suggests one compromised-target investigation, while spraying suggests the attacker may already hold several accounts' worth of partial information and is casting a wider net — a different, often more urgent, response.

5. Remediation Attempt — Custom Rule 100020

To distinguish spraying from brute force correctly, a new rule was written using Wazuh's `<different_user />` correlation tag — a dedicated option that requires each matched event in the frequency window to target a different account, rather than simply counting failures regardless of target:

```
<rule id="100020" level="10" frequency="5" timeframe="120">
  <if_matched_sid>5503</if_matched_sid>
  <program_name>vsftpd</program_name>
  <same_source_ip />
  <different_user />
  <description>FTP password spraying: multiple distinct accounts targeted from same
source</description>
  <group>authentication_failures,recon,</group>
  <mitre>
    <id>T1110.003</id>
  </mitre>
</rule>
```

Built on the identical base rule, frequency, and timeframe as 100017 — the only addition is `<different_user />` — and tagged with `T1110.003`, MITRE's specific Password Spraying sub-technique, distinct from `T1110.001` (Password Guessing) used elsewhere in this ruleset.

The rule was added via the dashboard's rule editor and the ruleset reloaded cleanly:

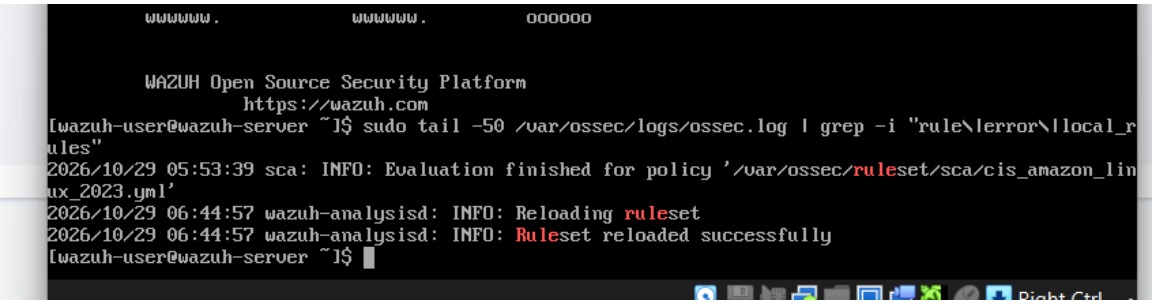


Figure 3: Wazuh manager log confirming a clean reload with no errors after adding rule 100020.

A direct check of the file on disk (`grep -A10 "100020" /var/ossec/etc/rules/local_rules.xml`) confirmed the rule was saved exactly as intended, properly closed, and sitting correctly inside the file's outer `<group>` wrapper — ruling out a save/sync issue as the cause of anything that followed.

6. Validation — A Documented Limitation

The spray was re-run (Run 2, single-threaded, fresh password) to generate clean traffic for the new rule. Discover confirmed, sorted chronologically, a textbook-clean sequence — six consecutive, entirely distinct usernames with zero repeats anywhere in the chain:

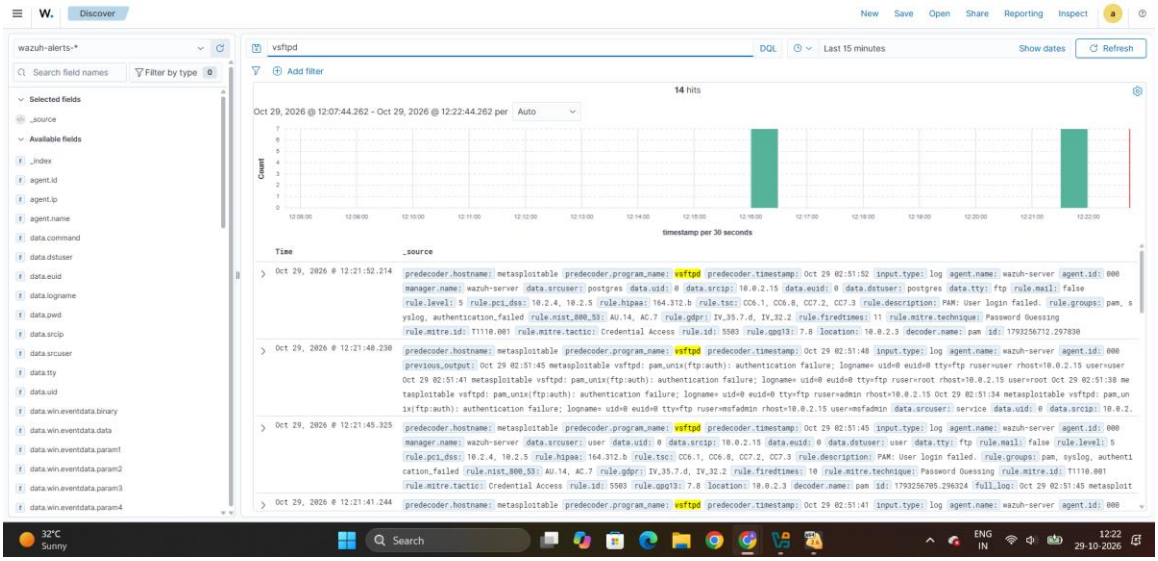


Figure 4: Discover, query `vsftpd`, showing `ruser=user`, `ruser=root`, `ruser=admin`, `ruser=msfadmin` and following entries — confirmed no duplicate username appears consecutively anywhere in the run, ruling out attack-tool retry behavior as an explanation for what follows.

Despite this, rule **100020 did not fire**. Checking `rule.id: 100020` directly in Discover over a 24-hour window returned zero results, at any severity — not merely absent from the `rule.level >= 10` alert view. Checking the rule's own `rule.groups` tag (`recon`) also returned nothing.

This was investigated methodically rather than assumed: the rule's syntax was confirmed correct and present on disk (Section 5); the ruleset reload was confirmed clean, with no option-level warnings in the manager log; the attack traffic was confirmed to be a genuinely clean, fully-distinct username sequence (above), ruling out the most likely explanation (duplicate usernames from Hydra connection retries breaking the distinct-value chain). With every other variable eliminated, the root cause appears to be specific to how Wazuh's correlation engine handles two independent frequency rules sharing an identical `if_matched_sid` + `same_source_ip` + `frequency`/`timeframe` signature, differing only by the addition of `<different_user />` — a scenario not covered in the troubleshooting performed for this exercise and left as an open, named limitation rather than a silently abandoned rule.

7. Findings & Conclusion

Scenario	Result
Password spray vs. 7-account list, fixed password	6+ distinct accounts probed from one source; confirmed via raw logs
Existing rule 100017 (brute-force rule)	Fired correctly at level 10, but description mislabels the activity as single-account brute force
Custom rule 100020 (`<different_user />`)	Correct syntax, confirmed saved and loaded, did not fire despite clean, fully-distinct attack traffic

This exercise is the first in this project where a planned remediation did not fully succeed, and it is documented that way deliberately rather than smoothed over. The underlying detection goal was still met — rule 100017 reliably flags this traffic as high-severity — but the finding that its *label* is misleading for a spray attack is itself a legitimate, reportable gap: an analyst triaging this alert would reasonably misjudge the scope of the incident.

The attempted fix, rule 100020, followed the same disciplined process used for every other rule in this project (confirm the base match, build the minimal addition, verify the save, verify the reload, test with real attack traffic) and still did not resolve the mislabeling. Rather than iterate further on an internal correlation-engine behavior with no clear documentation and no guaranteed payoff, this was treated as a stopping point worth recording honestly: not every detection gap closes cleanly on the first well-reasoned attempt, and knowing when to document a limitation instead of continuing to debug blind is itself part of a mature SOC workflow. A follow-up worth investigating separately: whether Wazuh requires `<different_user />` combined with `<same_user />` in a specific pairing, or whether two frequency rules sharing an identical base match need to be deconflicted via rule ordering or an `<if_group>` exclusion rather than coexisting independently.